Day 1 — ML Foundations and Mathematical Language

Objective

Build the vocabulary and mathematical intuition needed to understand why ML algorithms work.

Topics and detailed notes

- Machine learning as parameter estimation and generalization rather than pattern memorization.
- Supervised learning: inputs X and target y. Regression predicts a continuous quantity; classification predicts a class or probability.
- Unsupervised learning: structure is inferred without a supplied target.
- Features, target, observation, parameter, hyperparameter, estimator, loss, objective, inference, artifact.
- Training data is used to learn parameters. Validation data supports decisions about model configuration. The final test set estimates performance after decisions are frozen.
- Generalization: performance on unseen data from the intended deployment distribution.
- Baseline: the simplest meaningful reference against which a model must prove value.
- Batch inference vs online inference.
- Offline training vs online prediction.

Mathematics / formulas to know

Vector: an ordered set of values representing an observation or parameters.

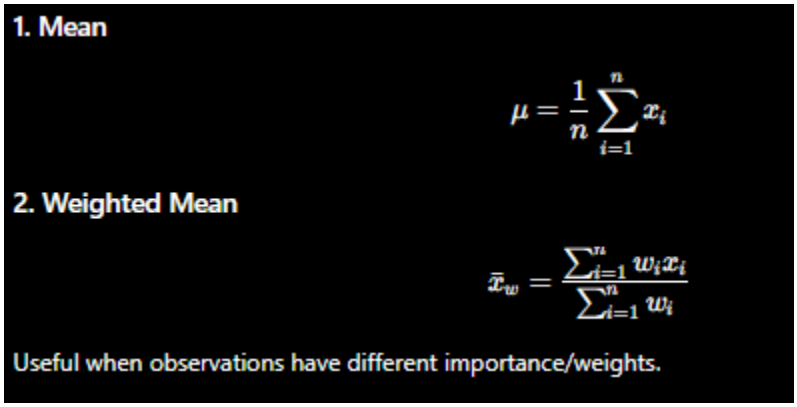
Matrix multiplication combines observations and coefficients; Xw produces one score per observation.

Dot product: a weighted sum of feature values.

Derivative: local rate of change. Gradient: vector of partial derivatives.

Gradient descent update: $\theta \leftarrow \theta - \alpha \nabla J(\theta)$.

Learning rate α controls the size of the update.



3. Variance

Population variance:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

Sample variance:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

4. Standard Deviation

$$\sigma = \sqrt{\sigma^2}$$

It measures the typical spread of values around the mean.

5. Z-Score

$$z = \frac{x - \mu}{\sigma}$$

This tells you how many standard deviations an observation is away from the mean.

6. Covariance

$$Cov(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

It describes how two variables vary together.

7. Pearson Correlation

$$r = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

with:

$$-1 \leq r \leq 1$$

8. Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Usually maps values into:

$$[0, 1]$$

9. Standardization

$$x' = \frac{x - \mu}{\sigma}$$

After standardization, the feature approximately has:

$$\mu = 0, \qquad \sigma = 1$$

10. Euclidean Distance

For two points:

$$x = (x_1, x_2, \ldots, x_n)$$

and

$$y = (y_1, y_2, \ldots, y_n)$$

the distance is:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

You'll need this later for algorithms such as **KNN and clustering**.

11. Basic Probability

$$P(A) = \frac{\text{favorable outcomes}}{\text{total outcomes}}$$

And conditional probability:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

12. Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

This becomes important later for **Naive Bayes**.

Implementation pattern

```
Problem → mathematical formulation → data → objective/loss → optimization → model →
evaluation → decision.
```

Experiments to perform

- Create a tiny synthetic regression dataset and manually compute a prediction from a weight vector.
- Calculate a dot product by hand and with NumPy.
- Implement a simple loss calculation.
- Implement one gradient-descent update manually before using a library implementation.

End-of-day deliverables

- Change the learning rate and observe convergence.
- Use poorly scaled features and observe optimization behavior.
- Compare a constant baseline with a learned model.

Mastery questions

1. One notebook explaining the ML vocabulary.
2. A small NumPy gradient-descent experiment.
3. A written explanation of training vs validation vs test.

PLAN

Welcome to Day 1. As your instructor and a Senior ML Engineer, I am thrilled to see this curriculum. Today is about laying the concrete foundation. Before we can build sophisticated models, we must establish a shared vocabulary and internalize the mathematical language of optimization.

We are not here to memorize patterns; machine learning is fundamentally about parameter estimation and generalization.

Here is our master plan and your specific tasks for the day, aligned with our Global ML Workflow.

Phase 1: The ML Vocabulary & Mental Models

Before touching any code, we must lock down the terminology. This phase directly maps to **Steps 1-3 of our Global ML Workflow** (Defining the problem, target, and success metric) and sets the stage for distinguishing between **Training-time vs Inference-time**.

- **Supervised vs. Unsupervised Learning:** Supervised learning involves inputs X and a target y, where regression predicts continuous quantities and classification predicts a class or probability. Unsupervised learning infers structure without a supplied target.
- **The Data Split:** Training data learns parameters, validation data supports model configuration decisions, and the final test set estimates performance after decisions are frozen.

- **The Baseline:** You must establish the simplest meaningful reference against which your model must prove its value.
- **Inference & Prediction Contexts:** We must differentiate between batch versus online inference, and offline training versus online prediction.

Phase 2: The Mathematics of Preprocessing & Optimization

This phase supports **Step 8 (Preprocessing)** and **Step 9 (Training)** of our workflow. When you start routing datasets through your FastAPI and Supabase architecture for automated exploratory data analysis, these scaling functions and probability theorems will be the exact mathematical transformations running inside your data cleaning pipelines.

Concept	Mathematical Formulation	Description
Mean	$\mu=\frac{1}{n}\sum_{i=1}^{n}x_{i}$	The standard average of a dataset.
Weighted Mean	$\overline{x}_{w}=\frac{\sum_{i=1}^{n}w_{i}x_{i}}{\sum_{i=1}^{n}w_{i}}$	Useful when observations have different importance/weights.
Sample Variance	$s^{2}=\frac{1}{n-1}\sum_{i=1}^{n}(x_{i}-\overline{x})^{2}$	Measures the typical spread of values around the mean for a sample.
Z-Score	$z=\frac{x-\mu}{\sigma}$	Tells you how many standard deviations an observation is away from the mean.
Pearson Correlation	$r=\frac{Cov(X,Y)}{\sigma_{X}\sigma_{Y}}$	Evaluates how variables move together, bounded by $-1\le r\le1$.

Min-Max Scaling	$x^{\prime}=\frac{x-x_{min}}{x_{max}-x_{min}}$	Usually maps values into the range [0, 1].
Standardization	$x^{\prime}=\frac{x-\mu}{\sigma}$	Forces the feature to approximately have $\mu=0$ and $\sigma=1$.
Euclidean Distance	$d(x,y)=\sqrt{\sum_{i=1}^{n}(x_{i}-y_{i})^{2}}$	Calculates distance between two points, needed for algorithms like KNN.
Bayes' Theorem	$P(A\vert{}B)=\frac{P(B\vert{}A)P(A)}{P(B)}$	Calculates conditional probability, important for Naive Bayes.
Gradient Descent	$\theta\leftarrow\theta-\alpha\nabla J(\theta)$	The update rule where learning rate α controls the size of the update.

Phase 3: Today's Tasks & Deliverables

This is where theory meets implementation. The required implementation pattern is strict: Problem $\rightarrow$ mathematical formulation $\rightarrow$ data $\rightarrow$ objective/loss $\rightarrow$ optimization $\rightarrow$ model $\rightarrow$ evaluation $\rightarrow$ decision.

Task 1: The ML Vocabulary Notebook (Deliverable 1)

- Create a clean, well-documented Jupyter Notebook defining every core term (Features, target, observation, parameter, hyperparameter, estimator, loss, objective, inference, artifact).

Task 2: The Train/Val/Test Strategy Document (Deliverable 3)

- Write a clear explanation of training vs validation vs test datasets.
- Explicitly map this to our Global ML Workflow **Step 6 (Decide the train/validation/test strategy)** and **Step 13 (Freeze the final test set until final evaluation)**.

Task 3: The NumPy Gradient Descent Engine (Deliverable 2)

- Create a tiny synthetic regression dataset.
- Manually compute a prediction from a weight vector.
- Calculate a dot product by hand and then verify it with NumPy.
- Implement a simple loss calculation.
- Implement one manual gradient-descent update before using any library implementations. Writing a manual gradient descent loop in NumPy requires the same strict algorithmic tracing and precision you use when verifying path costs in custom Python search algorithms to ensure they evaluate to exactly 6.

Task 4: Optimization Experiments

- Change the learning rate and observe how it affects convergence.
- Feed poorly scaled features into your model and observe the optimization behavior.
- Compare a constant baseline model against your learned model.